

Teach Module 32: Resilience and Fault Tolerance Patterns

Source document used only to identify the module topic: docs/Java Software Engineer bootcamp.docx .

Teaching note: This README teaches the topic independently and does not reuse the course material content.

Module Focus

Module 32 focuses on designing Java and Spring Boot services that can survive failure in distributed systems.

The main ideas are:

- Why distributed systems fail
- Timeouts
- Retries
- Exponential backoff
- Circuit breakers
- Fallback responses
- Bulkheads
- Rate limiting
- Applying resilience patterns in Spring Boot services

Core Explanation

When services talk to each other over a network, failure is normal. A payment service might be slow. A user service might restart. A database might briefly reject connections. Your job as a backend engineer is not to pretend failures will not happen, but to design your service so one failure does not bring down the whole system.

The core idea is:

Fail gracefully, recover automatically, and avoid making a bad situation worse.

In a distributed system, a simple request may cross several services:

Frontend -> Order API -> Inventory Service -> Payment Service -> Email Service

Any hop can fail because of:

- Network latency
- Service downtime
- Database overload
- Bad deployments
- Rate limits
- Thread pool exhaustion
- Slow third-party APIs

A common beginner mistake is writing code like this:

```
PaymentResponse response = paymentClient.charge(order);
```

and assuming it either works or throws quickly. In real systems, it may hang, respond slowly, partially fail, or repeatedly fail under load.

That is why we use resilience patterns.

Timeout

A timeout says:

```
I will only wait this long.
```

Without timeouts, one slow downstream service can consume all your server threads. Eventually your healthy service becomes unhealthy too.

Example with `WebClient` :

```
WebClient client = WebClient.builder()
    .baseUrl("http://payment-service")
    .build();

PaymentResponse response = client.post()
    .uri("/payments")
    .bodyValue(request)
    .retrieve()
    .bodyToMono(PaymentResponse.class)
    .timeout(Duration.ofSeconds(2))
    .block();
```

If the payment service does not respond within two seconds, the caller stops waiting.

Rule of thumb:

```
Every remote call should have a timeout.
```

Retry

A retry means:

```
If the operation fails, try again.
```

Retries are useful for temporary failures:

- Network hiccups
- Brief service restarts
- Temporary database locks
- HTTP 503 Service Unavailable

Retries can also be dangerous. If a service is already overloaded, thousands of clients retrying immediately can make the outage worse.

Bad retry behavior:

```
Try -> fail -> immediately retry -> fail -> immediately retry
```

Better retry behavior:

```
Try -> fail -> wait 200ms -> retry -> wait 500ms -> retry -> wait 1s
```

This is called exponential backoff.

Only retry operations that are safe to repeat.

Usually safe:

- GET /products/123
- Reading from another service

Risky:

- POST /payments
- POST /orders
- Sending emails

For risky operations, you need idempotency, meaning repeating the same request does not create duplicate side effects.

Circuit Breaker

A circuit breaker protects your system from repeatedly calling a failing dependency.

Think of it like an electrical circuit breaker. If too many calls fail, it opens and stops sending traffic temporarily.

It has three major states:

```
Closed -> normal calls allowed
Open -> calls blocked immediately
Half-open -> allow a few test calls to see if service recovered
```

Example flow:

```
Payment service starts failing
Order service keeps calling it
Failures pass threshold
Circuit opens
Order service stops calling payment service temporarily
After a wait period, circuit half-opens
If test calls succeed, circuit closes
If test calls fail, circuit opens again
```

Why this matters:

```
Without a circuit breaker, your service wastes time and resources calling something that is already failing.
```

Fallback

A fallback is an alternate response when the normal path fails.

Examples:

- If a recommendation service fails, return popular products.
- If a profile image service fails, show a default avatar.
- If a pricing service fails, return `price temporarily unavailable`.
- If a payment service fails, mark the order as `PAYMENT_PENDING`.

Fallbacks should be honest. Do not pretend the operation succeeded if it did not.

Bad fallback:

```
Payment failed, but return "payment successful".
```

Good fallback:

```
Payment service unavailable. Order saved with payment pending.
```

Bulkhead Pattern

A bulkhead isolates failures.

Imagine your app has two types of calls:

- Payment calls
- Product search calls

If both use the same thread pool, payment slowness can consume everything and break product search too.

With bulkheads, you isolate resources:

```
Payment thread pool: 20 threads
Search thread pool: 50 threads
Email thread pool: 10 threads
```

If payment breaks, search can still work.

Rate Limiting

Rate limiting controls how many requests are allowed in a given time.

Example:

```
Maximum 100 requests per user per minute
Maximum 1000 requests per service per second
```

This protects your system from abuse, bugs, traffic spikes, and retry storms.

Common Spring Boot Tooling

In Java and Spring Boot, a common library for resilience is Resilience4j.

It supports:

- Retry
- Circuit breaker
- Rate limiter
- Bulkhead
- Time limiter

Example:

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
@Retry(name = "paymentService")
public PaymentResponse chargePayment(PaymentRequest request) {
    return paymentClient.charge(request);
}

public PaymentResponse paymentFallback(PaymentRequest request, Exception ex) {
    return new PaymentResponse("PENDING", "Payment service temporarily unavailable");
}
```

This says:

- Try calling the payment service.
- Retry according to the configuration if it fails.
- If failures continue, the circuit breaker activates.
- If still unavailable, use fallback behavior.

Practical Design Rules

Use these rules in real projects:

- Always set timeouts for remote calls.
- Retry only temporary failures.
- Use exponential backoff.
- Avoid retrying unsafe operations unless idempotency is implemented.
- Add circuit breakers around unstable or critical dependencies.
- Provide meaningful fallback responses.
- Log failures with enough context.
- Monitor failure rate, latency, retry count, and circuit breaker state.
- Do not hide failures that business users need to know about.

Mini Exercise

Imagine an `OrderService` calls a `PaymentService`.

Design the resilience behavior:

```
Timeout: 2 seconds
Retry: 2 retries with exponential backoff
```

```
Circuit breaker: opens if 50% of recent calls fail
Fallback: save order as PAYMENT_PENDING
```

Expected behavior:

```
User places order
Payment service is slow
Order service waits max 2 seconds
Retries briefly
If still failing, stops calling payment service
Order is saved as PAYMENT_PENDING
User sees: "Order received. Payment is being processed."
```

That is resilient design: the user gets a controlled experience, the system does not collapse, and recovery can happen later.

Practice Exercises

Exercise 1: Add a Timeout

Create a Spring Boot service that calls a fake external API, such as a `PaymentService` .

Practice goal:

```
If the payment API takes longer than 2 seconds, stop waiting and return a controlled error.
```

What to build:

- `OrderController`
- `OrderService`
- `PaymentClient`
- Simulate slow payment response using `Thread.sleep(5000)`
- Add timeout behavior
- Return a response like:

```
{
  "status": "PAYMENT_TIMEOUT",
  "message": "Payment service did not respond in time"
}
```

Exercise 2: Retry Temporary Failures

Create an endpoint that fails the first two times, then succeeds on the third call.

Practice goal:

```
Use retry logic for temporary failures.
```

Example behavior:

```
Attempt 1 -> fails
Attempt 2 -> fails
Attempt 3 -> succeeds
```

Practice:

- Retry count
- Delay between retries
- Logging retry attempts
- Avoiding infinite retries

Exercise 3: Exponential Backoff

Improve the retry exercise so each retry waits longer.

Example:

```
Attempt 1 -> wait 500ms
Attempt 2 -> wait 1s
Attempt 3 -> wait 2s
```

Practice goal:

Understand why immediate retries can overload a failing service.

Exercise 4: Circuit Breaker

Create a service call that fails repeatedly. After enough failures, stop calling it temporarily.

Practice goal:

Move through closed, open, and half-open circuit breaker states.

Expected behavior:

```
First few calls -> service is called and fails
After threshold -> circuit opens
Next calls -> fail fast without calling service
After wait time -> allow test call
If test call succeeds -> circuit closes
```

Use this with a fake `InventoryService` or `PaymentService` .

Exercise 5: Fallback Response

When a downstream service fails, return a fallback response.

Example scenario:

Product recommendation service is unavailable.

Instead of failing the whole request, return:

```
{  
  "recommendations": ["Popular Java Book", "Spring Boot Guide", "Clean Code"]  
}
```

Practice goal:

Keep the user experience working even when a non-critical service fails.

Exercise 6: Payment Pending Workflow

Build an `OrderService` where payment failure does not cancel the order.

Practice goal:

Handle important business workflows gracefully.

Expected behavior:

```
User places order  
Payment service fails  
Order is saved with status PAYMENT_PENDING  
User receives controlled response
```

Possible statuses:

```
CREATED  
PAYMENT_PENDING  
PAYMENT_CONFIRMED  
PAYMENT_FAILED
```

Exercise 7: Idempotent Retry

Practice making retries safe.

Scenario:

A payment request is retried.

Problem:

Without idempotency, the customer may be charged twice.

What to build:

- Send an `idempotencyKey` with each payment request.
- Store processed keys.
- If the same key appears again, return the original result.

- Do not process the payment twice.

Exercise 8: Bulkhead Simulation

Create separate thread pools for two operations:

```
Payment processing
Product search
```

Practice goal:

```
Payment failures should not block product search.
```

Simulate payment being slow, then verify product search still responds normally.

Exercise 9: Rate Limiting

Add a limit to an endpoint.

Example:

```
Maximum 5 requests per user per minute
```

Practice goal:

```
Protect an API from too many requests.
```

Expected response after the limit is exceeded:

```
{
  "status": "RATE_LIMIT_EXCEEDED",
  "message": "Too many requests. Try again later."
}
```

Exercise 10: Resilience4j Integration

Use Resilience4j in a Spring Boot app.

Practice these annotations:

```
@Retry
@CircuitBreaker
@RateLimiter
@Bulkhead
@TimeLimiter
```

Recommended mini-project:

```
Order API calls Payment API and Inventory API.
```

Add timeout, retry, circuit breaker, and fallback handling.

Lab: Resilient Order Processing API

Goal

Build a Spring Boot API that handles failures using timeout, retry, circuit breaker, and fallback behavior.

Scenario

You are building an `OrderService`. When a customer places an order, your app must:

1. Check inventory.
2. Process payment.
3. Save the order.
4. Return a useful response even when another service fails.

You will simulate failures instead of using real external services.

Lab Requirements

Create a Spring Boot project with these endpoints:

```
POST /orders
GET /orders/payments/health-mode/{mode}
GET /orders/inventory/health-mode/{mode}
```

Use these fake modes:

```
UP
SLOW
DOWN
UNSTABLE
```

Expected inventory behavior:

```
UP -> returns success
SLOW -> delays response
DOWN -> always fails
UNSTABLE -> fails randomly
```

Expected payment behavior:

```
UP -> payment succeeds
SLOW -> causes timeout
DOWN -> payment fails
UNSTABLE -> sometimes succeeds, sometimes fails
```

Step 1: Create Order Request

```

public class OrderRequest {
    private String customerId;
    private String productId;
    private int quantity;
    private BigDecimal amount;

    // getters and setters
}

```

Step 2: Create Order Response

```

public class OrderResponse {
    private String orderId;
    private String status;
    private String message;

    public OrderResponse(String orderId, String status, String message) {
        this.orderId = orderId;
        this.status = status;
        this.message = message;
    }

    // getters and setters
}

```

Step 3: Add Order Statuses

```

public enum OrderStatus {
    CREATED,
    INVENTORY_UNAVAILABLE,
    PAYMENT_CONFIRMED,
    PAYMENT_PENDING,
    PAYMENT_FAILED
}

```

Step 4: Simulate Inventory Service

```

@Service
public class InventoryClient {

    private String mode = "UP";

    public void setMode(String mode) {
        this.mode = mode;
    }

    public boolean checkInventory(String productId, int quantity) {
        if ("DOWN".equalsIgnoreCase(mode)) {

```

```

        throw new RuntimeException("Inventory service is down");
    }

    if ("SLOW".equalsIgnoreCase(mode)) {
        sleep(5000);
    }

    if ("UNSTABLE".equalsIgnoreCase(mode) && Math.random() < 0.5) {
        throw new RuntimeException("Temporary inventory failure");
    }

    return true;
}

private void sleep(long milliseconds) {
    try {
        Thread.sleep(milliseconds);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}

```

Step 5: Simulate Payment Service

```

@Service
public class PaymentClient {

    private String mode = "UP";

    public void setMode(String mode) {
        this.mode = mode;
    }

    public boolean charge(String customerId, BigDecimal amount) {
        if ("DOWN".equalsIgnoreCase(mode)) {
            throw new RuntimeException("Payment service is down");
        }

        if ("SLOW".equalsIgnoreCase(mode)) {
            sleep(5000);
        }

        if ("UNSTABLE".equalsIgnoreCase(mode) && Math.random() < 0.5) {
            throw new RuntimeException("Temporary payment failure");
        }

        return true;
    }

    private void sleep(long milliseconds) {

```

```

        try {
            Thread.sleep(milliseconds);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}

```

Step 6: Add Resilience4j Dependency

For Maven:

```

<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>

```

Step 7: Configure Resilience4j

In `application.yml` :

```

resilience4j:
  retry:
    instances:
      inventoryRetry:
        max-attempts: 3
        wait-duration: 1s

  circuitbreaker:
    instances:
      paymentCircuitBreaker:
        failure-rate-threshold: 50
        minimum-number-of-calls: 4
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2

  timelimiter:
    instances:
      paymentTimeout:
        timeout-duration: 2s

```

Step 8: Create Order Service

```

@Service
public class OrderService {

```

```

private final InventoryClient inventoryClient;
private final PaymentClient paymentClient;

public OrderService(InventoryClient inventoryClient, PaymentClient paymentClient) {
    this.inventoryClient = inventoryClient;
    this.paymentClient = paymentClient;
}

@Retry(name = "inventoryRetry", fallbackMethod = "inventoryFallback")
public boolean checkInventory(OrderRequest request) {
    return inventoryClient.checkInventory(
        request.getProductId(),
        request.getQuantity()
    );
}

public boolean inventoryFallback(OrderRequest request, Exception ex) {
    return false;
}

@CircuitBreaker(name = "paymentCircuitBreaker", fallbackMethod = "paymentFallback")
public boolean processPayment(OrderRequest request) {
    return paymentClient.charge(
        request.getCustomerId(),
        request.getAmount()
    );
}

public boolean paymentFallback(OrderRequest request, Exception ex) {
    return false;
}

public OrderResponse placeOrder(OrderRequest request) {
    String orderId = UUID.randomUUID().toString();

    boolean inventoryAvailable = checkInventory(request);

    if (!inventoryAvailable) {
        return new OrderResponse(
            orderId,
            "INVENTORY_UNAVAILABLE",
            "Order could not be completed because inventory is unavailable."
        );
    }

    boolean paymentSuccessful = processPayment(request);

    if (!paymentSuccessful) {
        return new OrderResponse(
            orderId,
            "PAYMENT_PENDING",

```

```

        "Order was created, but payment is pending."
    );
}

return new OrderResponse(
    orderId,
    "PAYMENT_CONFIRMED",
    "Order placed successfully."
);
}
}

```

Step 9: Create Controller

```

@RestController
@RequestMapping("/orders")
public class OrderController {

    private final OrderService orderService;
    private final InventoryClient inventoryClient;
    private final PaymentClient paymentClient;

    public OrderController(
        OrderService orderService,
        InventoryClient inventoryClient,
        PaymentClient paymentClient
    ) {
        this.orderService = orderService;
        this.inventoryClient = inventoryClient;
        this.paymentClient = paymentClient;
    }

    @PostMapping
    public OrderResponse placeOrder(@RequestBody OrderRequest request) {
        return orderService.placeOrder(request);
    }

    @GetMapping("/inventory/health-mode/{mode}")
    public String setInventoryMode(@PathVariable String mode) {
        inventoryClient.setMode(mode);
        return "Inventory mode set to " + mode;
    }

    @GetMapping("/payments/health-mode/{mode}")
    public String setPaymentMode(@PathVariable String mode) {
        paymentClient.setMode(mode);
        return "Payment mode set to " + mode;
    }
}

```

Step 10: Test the Lab

Set services to healthy:

```
GET /orders/inventory/health-mode/UP
GET /orders/payments/health-mode/UP
```

Place an order:

```
POST /orders
```

Body:

```
{
  "customerId": "C1001",
  "productId": "P2001",
  "quantity": 2,
  "amount": 49.99
}
```

Expected:

```
{
  "status": "PAYMENT_CONFIRMED",
  "message": "Order placed successfully."
}
```

Now make payment fail:

```
GET /orders/payments/health-mode/DOWN
```

Call `POST /orders` multiple times.

Expected:

```
{
  "status": "PAYMENT_PENDING",
  "message": "Order was created, but payment is pending."
}
```

Now make inventory unstable:

```
GET /orders/inventory/health-mode/UNSTABLE
```

Call `POST /orders` several times and observe retry behavior.

Lab Challenge

Add logging so you can see:

```
Checking inventory...
Retrying inventory...
Processing payment...
Payment fallback triggered...
Order saved as PAYMENT_PENDING
```

Success Criteria

You completed the lab if:

- Inventory failures do not crash the API.
- Payment failures return `PAYMENT_PENDING`.
- Retry is used for inventory.
- Circuit breaker is used for payment.
- The API always returns a controlled response.
- You can explain why retrying payment blindly can be dangerous.

Key Takeaway

Resilience is not about preventing every failure. It is about making failure boring, contained, and recoverable. In enterprise Java systems, patterns like timeouts, retries, circuit breakers, fallbacks, bulkheads, and rate limits are what keep one broken dependency from turning into a full outage.